

1. 개요

주어진 Data Set에 대해 DBSCAN을 통해 밀도 기반 클러스터링을 수행한다. 생성된 클러스터를 기반으로 두 오브젝트가 같은 클러스터에 있는지 판단하고, 그 결과를 제공된 결과 파일과 비교하여 클러스터링의 정확도를 평가한다.

2. 알고리즘 요약

가. DBSCAN

DBSCAN은 밀도 기반 클러스터링의 대표적인 기법으로, 거리 반경인 Eps와 반경 내 최소 점의 개수인 MinPts 2개의 parameter를 사용한다. 밀도 기반 클러스터링에서는 클러스터를 density-connected point들의 maximal set으로 정의해 클러스터링을 수행 한다.

1) Directly density-reachable

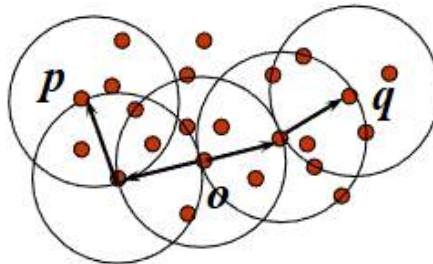
점 p 가 점 q 로부터 density-reachable 하려면, p 가 q 로부터 Eps거리 안에 있어야 하며, q 에서 Eps 거리 안에 있는 점의 개수가 $|N_{Eps}(q)| \geq MinPts$ 를 만족해야 한다.

2) Density-reachable

주어진 Eps와 MinPts에서 p_{i+1} 이 p_i 로부터 Directly density-reachable한 chain $p_1, ..., p_n$ 이 형성되면, p_n 은 p_1 으로부터 density-reachable하다.

3) Density-connected

주어진 Eps와 MinPts에서 점 O 가 존재해 p 와 q 가 모두 O 로부터 density-reachable 하면 p 와 q 는 서로 Density-connected 관계에 있다.



<그림 1> Density-connected

4) 알고리즘 설명

DBSCAN에서는 점을 다음과 같이 구분한다.

가) Core Point

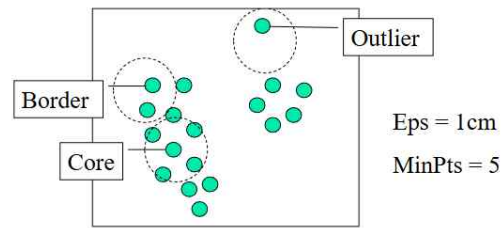
Eps 내에 자신 포함 MinPts 개 이상의 점을 가진 중심점.

나) Border Point

Core Point은 아니지만, Core Point의 Eps 내에 속하는 점.

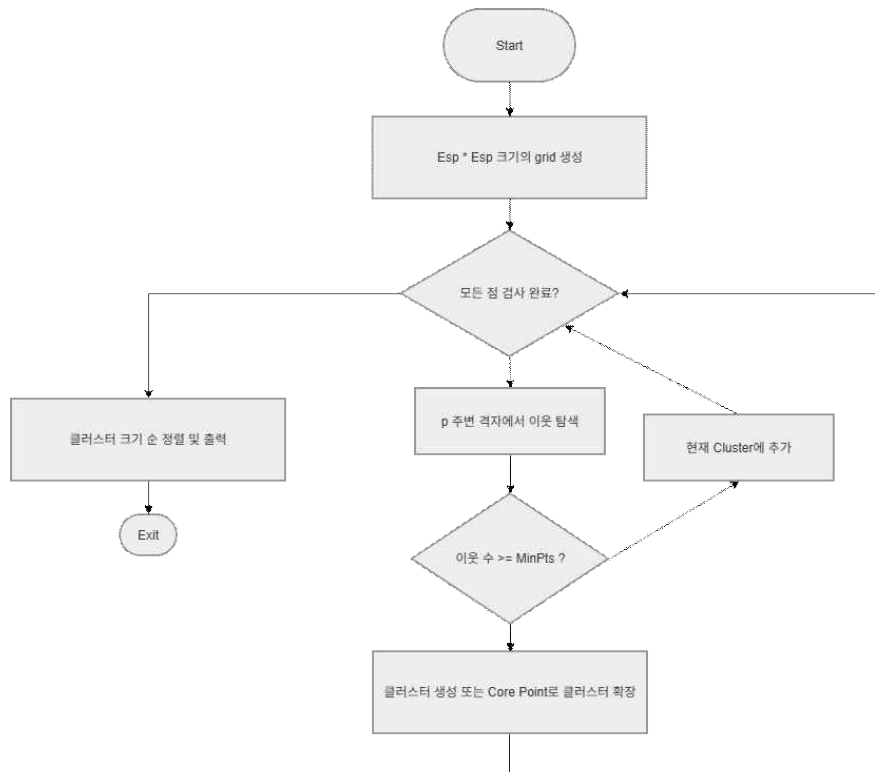
우선 임의의 점 p 를 선정하여 p 가 코어 포인트면 클러스터를 형성한다. 이후 p 에서 Eps 내에 존재하는 점 q 에 대해 만약 q 가 Border Point라면 같은 클러스터에 포함 시키고, Core

Point라면 q를 중심으로 해, 주변 점에 같은 test를 진행해 현재 클러스터를 확장 시킨다.



<그림 2> Border Point와 Core Point

나. Flow Chart



<그림 3> DBSCAN 과정

Point를 검사할 때마다. 주변 Eps 거리 내에 몇 개의 점이 있는지 검사해야 하는데 이를 전체 검사하면 $O(n^2)$ 이라는 비효율적인 연산을 해야 한다. 이를 해결하기 위해, 전체 공간을 한칸 Esp*Esp 크기의 grid로 분할하여, 상하좌우 9칸만 확인하면 되도록 미리 grid를 구성한다. 이후 임의의 점을 대상으로, 주변 grid를 확인해 Eps 거리 내에, MinPts 이상의 점이 있으면 클러스터를 생성하거나 현재 클러스터의 Core Point로 현재 클러스터를 계속 확장한다. MinPts 미만이라면 현재 클러스터에 추가하고, 모든 점을 확인할 때까지 이를 반복 한다. 만약 해당 과정에서 어떤 클러스터에도 들어가지 못하는 점은 outlier로 간주된다.

모든 점을 검사한 이후에는 클러스터 크기 순으로 정렬해 상위 n개를 파일로 출력한다.

3. 주요 함수 및 코드 설명

가. Dbscan

1) grid 구축

```
def Dbscan(points, eps, minPts):
    grid = {}
    for pt in points:
        gx = int(pt[1] / eps)
        gy = int(pt[2] / eps)
        gridKey = (gx, gy)
        if gridKey not in grid:
            grid[gridKey] = []
        grid[gridKey].append(pt)
```

<그림 4> grid 구축

Eps 안의 이웃 개수를 세는 연산을 줄이기 위해 grid를 먼저 생성한다. 점의 좌표를 eps로 나눠 몇 번째 칸에 들어갈지를 gridKey로 하고, 해당 칸에 들어갈 점을 list로 저장한다. gridKey와 이에 해당하는 list를 묶어 grid라는 dictionary로 저장한다.

2) 초기 CorePoint 탐색 및 Cluster 생성

```
visited = set()
clustered = set()
clusters = []

for pt in points:
    ptId = pt[0]
    if ptId in visited:
        continue
    visited.add(ptId)

    neighbors = []
    gx = int(pt[1] / eps)
    gy = int(pt[2] / eps)
    for dx in (-1, 0, 1):
        for dy in (-1, 0, 1):
            gridKey = (gx + dx, gy + dy)
            if gridKey in grid:
                for cand in grid[gridKey]:
                    dist = ((pt[1] - cand[1])**2 + (pt[2] - cand[2])**2)**0.5
                    if dist <= eps:
                        neighbors.append(cand[0])

    if len(neighbors) < minPts:
        continue

    currentCluster = [ptId]
    clustered.add(ptId)
```

<그림 5> 초기 CorePoint 탐색 및 Cluster 생성

임의의 점에 대해 Eps 내에 있는 이웃이 몇 개인지 확인한다. 이때 grid를 통해 상하좌우 9개의 칸 안에 있는 점에 대해서만 확인한다. 만약 Eps 내 이웃한 점이 MinPts 이상이라면 해당 점을 Corepoint로 간주하고, 새로운 클러스터를 형성하고, 해당 클러스터에 추가한다.

3) 클러스터 탐색

```
seedSet = list(neighbors)
if ptId in seedSet:
    seedSet.remove(ptId)
for nId in seedSet:
    if nId not in clustered:
        clustered.add(nId)
        currentCluster.append(nId)
idx = 0
while idx < len(seedSet):
    currId = seedSet[idx]
    if currId not in visited:
        visited.add(currId)

        currPt = pointDict[currId]
        currNeighbors = []
        cgx = int(currPt[1] / eps)
        cgy = int(currPt[2] / eps)
        for dx in (-1, 0, 1):
            for dy in (-1, 0, 1):
                gridKey = (cgx + dx, cgy + dy)
                if gridKey in grid:
                    for cand in grid[gridKey]:
                        dist = ((currPt[1] - cand[1])**2 + (currPt[2] - cand[2])**2)
                        if dist <= eps:
                            currNeighbors.append(cand[0])

        idx += 1
clusters.append(currentCluster)
return clusters
```

<그림 6> 클러스터 탐색

Core Point에서 Eps 내에 있는 이웃들을 seedSet에 추가한다. 만약 그 점이 아직 클러스터링 되지 않았다면 현재 클러스터에 추가한다. seedSet에 있는 점들에 대해 처음과 같은 방법으로 Eps 거리 내에 있는 이웃을 상하좌우 grid를 통해 currNeighbors에 추가한다.

4) 클러스터 확장

```
if len(currNeighbors) >= minPts:
    for nId in currNeighbors:
        if nId not in seedSet:
            seedSet.append(nId)
        if nId not in clustered:
            clustered.add(nId)
            currentCluster.append(nId)
    idx += 1
clusters.append(currentCluster)
return clusters
```

<그림 7> 클러스터 확장

만약 해당 점의 Eps거리 내에 MinPts개 이상의 이웃한 점이 있다면, 해당 점을 Eps의 새로운 Core Point로 간주한다. 새로운 Core Point라면 해당 점의 이웃들을 다시 seedSet과 현재 클러스터에 추가하여 클러스터를 확장한다.

seedSet을 전부 탐색했다면 해당 클러스터를 클러스터리스트에 추가하고, 아직 탐색하지 않은 점들을 추가로 탐색한다. 이후 모든 점을 탐색했다면, 지금까지 발견한 클러스터 리스트를 반환한다.

나. Main

1) 입력 파일 파싱

```
def main():
    if len(sys.argv) < 5:
        sys.exit(1)

    inputFileName = sys.argv[1]
    n = int(sys.argv[2])
    eps = float(sys.argv[3])
    minPts = int(sys.argv[4])

    points = []
    with open(inputFileName, 'r') as file:
        for line in file:
            parts = line.strip().split('\t')
            if len(parts) >= 3:
                ptId = int(parts[0])
                x = float(parts[1])
                y = float(parts[2])
                points.append((ptId, x, y))
```

<그림 8> 입력 파일 파싱

인자로 입력 파일 이름, 클러스터의 수, Eps, MinPts를 받아 저장한다. 파일을 한 줄씩 읽어 '\t'로 구분된 ID, x좌표, y좌표를 튜플로 하나의 점으로 저장한다.

2) DBSCAN 진행 및 출력

```
clusters = Dbscan(points, eps, minPts)

baseName = os.path.splitext(os.path.basename(inputFileName))[0]
clusters.sort(key=len, reverse=True)
selected = clusters[:n]
for idx, cluster in enumerate(selected):
    cluster.sort()
    outFileName = f"{baseName}_cluster_{idx}.txt"
    with open(outFileName, 'w') as outFile:
        for ptId in cluster:
            outFile.write(f"{ptId}\n")

if __name__ == "__main__":
    main()
```

<그림 9> DBSCAN 진행 및 출력

DBSCAN을 진행 후, 만들어진 클러스터를 크기 기준 내림차순으로 정렬해 상위 n개의 클러스터만 선택한다. 이후 각각의 클러스터 내의 점들을 ID를 기준으로 정렬해, 주어진 출력 이름 형식에 맞게 파일 이름을 설정하고 클러스터마다 점의 번호를 한 줄씩 출력한다.

4. 실행 및 테스트

1) 실행환경

OS : Window, Python Version : Python 3.12.2

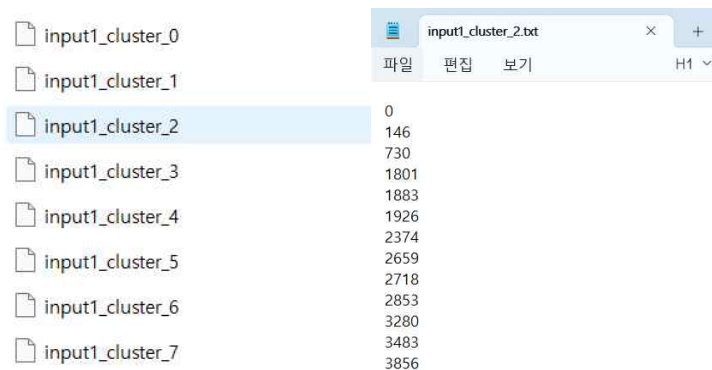
2) 실행 방법

```
python clustering.py input1.txt 8 15 22
```

<그림 10> 실행 방법

clustering.py 파일과 input 파일이 있는 경로에서 ‘python clustering.py [데이터파일명] [n] [Eps] [MinPts]’ 형식으로 실행하시면 됩니다. 기본적으로 clustering.py와 입출력파일이 같은 경로에 있는 것을 가정합니다.

3) 실행 결과



input1_cluster_0	
input1_cluster_1	
input1_cluster_2	0
input1_cluster_3	146
input1_cluster_4	730
input1_cluster_5	1801
input1_cluster_6	1883
input1_cluster_7	1926
	2374
	2659
	2718
	2853
	3280
	3483
	3856

<그림 11> 실행결과

인자로 주어진대로 8개의 클러스터가 나왔으며, 엔터로 구분된 점들의 번호가 각 클러스터마다 작성되었다.

4) 테스트

```
./PA3.exe input1
```

<그림 12> 테스트 방법

제공된 test 프로그램을 사용해 구현한 DBSCAN의 정확도를 측정한다. 측정 방법은 임의의 두 점에 대해, 같은 클러스터에 있는지에 대한 여부를 제공된 프로그램의 정답과 비교하여 (정답수) / (가능한 pair의 수)로 측정한다. ‘./PA3.exe [input파일 이름]’의 형식으로 진행한다.

```
PS C:\Users\HJ\Desktop\테사\sp3_DBSCAN\test-1> ./PA3.exe input1
98.97037점
```

<그림 13> 테스트 결과

	목표점수	획득점수
input1	99	98.97
input2	95	94.87
input3	99	99.98

<표 1> test 결과

3개의 input에서 모두 목표치에 근접한 점수를 획득했다. parameter는 명세에서 제공된 parameter를 각각 사용했다.

5. 결론

DBSCAN을 통해 밀도 기반 클러스터링을 진행했으며, 한 칸이 Eps*Eps 크기인 grid를 통해 주변만 탐색할 수 있도록 해 최적화를 진행하였다. 과제의 명세대로 입력으로 n, Eps, MinPts를 인자로 받아 클러스터링을 진행했으며, 출력 파일 형식에 맞게 파일 이름을 설정하고 내용을 출력했다. 제공된 테스트 파일로 정답과 비교하여 테스트한 결과 전부 목표치에 근접한 점수를 보여주어 명세의 요구대로 구현했음을 확인할 수 있었다.